# UTS

# PEMBELAJARAN MESIN




NAMA : Gary Patar Immanuel Silaban

NIM : 202431165

KELAS : H

DOSEN : Rizqia Cahyaningtyas, ST, M.Kom.

NO.PC : 12

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Ardy

3.

4.

# INSTITUT TEKNOLOGI PLN

# TEKNIK INFORMATIKA

# 2026/2027

## DAFTAR ISI

BAB I PENDAHULUAN .....1
1.1 Rumusan Masalah .....1
1.2 Tujuan Masalah .....1
1.3 Manfaat Masalah .....2
BAB II LANDASAN TEORI .....3
2.1 Pembelajaran Mesin (*Machine Learning*) dan Klasifikasi .....3
2.2 *Exploratory Data Analysis* (EDA) .....3
2.3 Pra-Pemrosesan Data (*Data Preprocessing*) .....3
2.4 Algoritma *Decision Tree* .....4
2.5 Algoritma *Naive Bayes* .....4
2.6 Evaluasi Kinerja Model .....5
BAB III HASIL .....6
3.1 Pemahaman Data dan Inspeksi Awal (*Data Loading*) ..... 6
3.2 *Exploratory Data Analysis* (EDA) .....7
3.3 Pra-Pemrosesan Data (*Data Preprocessing*) .....9
3.4 Implementasi dan Evaluasi Model 1: *Decision Tree* .....10
3.5 Implementasi dan Evaluasi Model 2: *Naive Bayes*.....11
3.6 Kesimpulan Hasil Klasifikasi .....12
BAB IV PENUTUP .....13
4.1 Kesimpulan .....13
DAFTAR PUSTAKA .....14

# BAB I

# PENDAHULUAN

**1.1 Rumusan Masalah** Berdasarkan latar belakang dan tugas praktikum Ujian Tengah Semester (UTS) Pembelajaran Mesin yang telah dilakukan pada *dataset Bank Customers*, maka rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana cara melakukan tahapan *Exploratory Data Analysis* (EDA) dan *preprocessing* data yang tepat (termasuk *encoding* dan *scaling*) agar *dataset* profil nasabah bank siap diproses oleh algoritma pembelajaran mesin?
2. Bagaimana cara mengimplementasikan dan melatih algoritma *Decision Tree* dan *Naive Bayes* untuk melakukan klasifikasi terhadap potensi nasabah dalam mengambil deposito berdasarkan fitur-fitur yang ada?
3. Bagaimana perbandingan performa, tingkat akurasi, dan matriks evaluasi dari hasil olahan data antara model *Decision Tree* dan *Naive Bayes* dalam menangani *dataset* yang memiliki ketidakseimbangan kelas (*class imbalance*)?

**1.2 Tujuan Masalah** Sesuai dengan rumusan masalah di atas, tujuan dan hasil yang telah dicapai dari pengerjaan UTS praktikum ini adalah:

1. Berhasil mengidentifikasi karakteristik data melalui *Exploratory Data Analysis* (EDA) dan menerapkan teknik *preprocessing* data, seperti *Label Encoding* untuk data biner, *One-Hot Encoding* untuk data multi-kategori, serta *Standard Scaling* untuk normalisasi nilai numerik.
2. Berhasil merancang, melatih, dan mengeksekusi model klasifikasi *Machine Learning* menggunakan algoritma *Decision Tree* dan *Gaussian Naive Bayes* menggunakan pustaka *Scikit-Learn* berbasis bahasa pemrograman Python.
3. Mendapatkan hasil akurasi evaluasi model, di mana *Decision Tree* menghasilkan akurasi sebesar 87,39% dan *Naive Bayes* menghasilkan akurasi sebesar 85,69%. Serta, mampu memberikan analisis mendalam mengenai kekuatan dan kelemahan masing-masing model ketika dihadapkan pada target prediksi data perbankan yang didominasi oleh kelas mayoritas (nasabah yang tidak mengambil deposito).

**1.3 Manfaat Masalah** Manfaat yang diperoleh dari pelaksanaan UTS praktikum dan penyusunan laporan ini meliputi beberapa aspek, yaitu:

1. **Manfaat Akademis dan Praktis:** Meningkatkan kemampuan teknis mahasiswa dalam menyusun *pipeline* pembelajaran mesin secara *end-to-end*, mulai dari pembersihan data mentah, rekayasa fitur (*feature engineering*), hingga penulisan kode pelatih model dan evaluasi matriks menggunakan Jupyter Notebook.

2. **Manfaat Teoritis:** Memperdalam pemahaman komparatif mengenai cara kerja fundamental algoritma *machine learning*. Mahasiswa dapat membuktikan secara langsung bagaimana algoritma berbasis aturan pemisahan (*Decision Tree*) bekerja menghadapi data rasio konversi dibandingkan dengan algoritma berbasis probabilitas teorema statistik (*Naive Bayes*).

3. **Manfaat Implementasi Bisnis (Konteks Data):** Memberikan wawasan (*insight*) tentang bagaimana pemodelan *machine learning* dapat diaplikasikan di industri perbankan nyata. Dengan model yang telah dibuat, pihak bank dapat memprediksi nasabah mana yang memiliki probabilitas tinggi untuk berlangganan deposito, sehingga bank dapat mengoptimalkan efisiensi biaya dan waktu dalam melakukan kampanye pemasaran (*marketing campaign*).

# BAB II

# LANDASAN TEORI

**2.1 Pembelajaran Mesin (*Machine Learning*) dan Klasifikasi** Pembelajaran mesin (*Machine Learning*) merupakan salah satu cabang dari kecerdasan buatan yang berfokus pada pengembangan algoritma untuk memungkinkan komputer belajar dari data tanpa harus diprogram secara eksplisit. Pendekatan ini secara garis besar dibagi menjadi tiga kategori utama: *Supervised Learning*, *Unsupervised Learning*, dan *Reinforcement Learning* (Sarker, 2021). Dalam konteks penyelesaian dataset profil nasabah bank, teknik yang digunakan tergolong dalam *Supervised Learning*, khususnya tugas klasifikasi. Klasifikasi adalah proses penemuan model (atau fungsi) yang mendeskripsikan dan membedakan kelas data atau konsep yang bertujuan untuk memprediksi kelas dari objek yang label kelasnya tidak diketahui. Model klasifikasi memetakan ruang fitur (variabel independen) ke dalam ruang label diskrit (variabel dependen).

**2.2 *Exploratory Data Analysis* (EDA)***Exploratory Data Analysis* (EDA) adalah pendekatan analitik awal untuk memahami karakteristik utama dari suatu dataset, seringkali menggunakan metode visualisasi grafis dan ringkasan statistik kuantitatif. Tujuan utama dari EDA adalah mendeteksi anomali, mengidentifikasi pola yang mendasarinya, menguji asumsi hipotesis, dan memeriksa distribusi variabel target (Milo & Silva, 2023). Dalam kasus prediksi deposito nasabah, EDA sangat esensial untuk mengetahui tingkat ketidakseimbangan kelas (*class imbalance*) pada variabel target. Ketidakseimbangan ini penting untuk diidentifikasi di awal karena dapat memicu bias pada model klasifikasi, di mana model akan cenderung over-prediksi pada kelas mayoritas dan mengabaikan kelas minoritas.

**2.3 Pra-Pemrosesan Data (*Data Preprocessing*)** Data mentah yang diperoleh dari dunia nyata seringkali mengandung *noise*, nilai yang hilang (*missing values*), atau format yang tidak konsisten sehingga tidak dapat diproses langsung oleh algoritma *machine learning*. Oleh karena itu, *data preprocessing* adalah tahap kritis yang sangat memengaruhi akurasi akhir dari model (Hasan et al., 2022). Terdapat beberapa tahapan utama dalam pra-pemrosesan data kategorikal dan numerikal:

1. ***Label Encoding* dan *One-Hot Encoding***: Sebagian besar algoritma prediktif beroperasi menggunakan komputasi matematis yang hanya dapat memproses nilai numerik. Untuk fitur biner atau ordinal, *Label Encoding* digunakan untuk mengonversi string menjadi nilai 0 dan 1. Sementara itu, untuk data kategorikal nominal yang memiliki lebih dari dua nilai unik (seperti jenis pekerjaan atau bulan), *One-Hot Encoding* diterapkan. Metode ini merepresentasikan data dengan menciptakan representasi matriks biner baru (vektor *dummy*) guna mencegah algoritma mengasumsikan adanya urutan hierarkis yang salah pada variabel nominal.

2. ***Standardisasi (Standard Scaling)***: Skala data yang bervariasi secara drastis antar fitur dapat mendistorsi fungsi objektif dari algoritma. *Standard Scaling* (Z-score normalization) menskalakan data sedemikian rupa sehingga memiliki nilai rata-rata (*mean*) sama dengan 0 dan standar deviasi sama dengan 1. Rumus matematis standardisasi adalah $z = \frac{x-\mu}{\sigma}$. Standardisasi ini sangat krusial bagi algoritma probabilistik dan berbasis jarak, seperti *Gaussian Naive Bayes*, untuk menjaga stabilitas fungsi kepadatan probabilitas (Kaur & Singh, 2024).

**2.4 Algoritma *Decision Tree***

*Decision Tree* (Pohon Keputusan) adalah salah satu algoritma klasifikasi *supervised learning* yang memecah dataset menjadi subhimpunan yang lebih kecil secara rekursif berdasarkan aturan keputusan (*decision rules*). Struktur ini terdiri dari *root node* (node akar), *internal nodes* (node percabangan yang merepresentasikan pengujian pada suatu atribut), dan *leaf nodes* (daun yang merepresentasikan hasil label kelas) (Charbuty & Abdulazeez, 2021). Algoritma ini bekerja dengan memilih atribut terbaik untuk memisahkan data menggunakan metrik evaluasi seperti *Gini Impurity* atau *Information Gain* (berbasis Entropi). Entropi dihitung menggunakan formula:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Kelebihan utama dari *Decision Tree* adalah interpretabilitasnya yang tinggi dan kemampuannya untuk memodelkan hubungan non-linear tanpa memerlukan asumsi distribusi parametrik dari data. Namun, algoritma ini sangat rentan terhadap *overfitting* apabila pohon dibiarkan tumbuh terlalu dalam tanpa dilakukan pemangkasan (*pruning*).

**2.5 Algoritma *Naive Bayes***

*Naive Bayes* adalah klasifikator probabilistik yang didasarkan pada penerapan Teorema Bayes dengan asumsi "naif" bahwa semua atribut fitur saling independen atau tidak memiliki ketergantungan satu sama lain jika diberikan label kelas (Berrar, 2023). Teorema Bayes dirumuskan sebagai berikut:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Di mana $P(Y|X)$ adalah probabilitas posterior, $P(X|Y)$ adalah *likelihood*, dan $P(Y)$ adalah probabilitas prior. Dalam praktikum ini, varian yang digunakan adalah *Gaussian Naive Bayes* (GaussianNB). Varian ini mengasumsikan bahwa fitur numerik kontinu mengikuti distribusi normal (Gaussian). *Likelihood* fitur direpresentasikan dengan Fungsi Kepadatan Probabilitas (PDF) Gaussian:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

Meskipun asumsi independensinya sering kali tidak terpenuhi di dunia nyata, *Naive Bayes* terbukti sangat cepat secara komputasi dan tetap berkinerja baik pada kondisi klasifikasi multikelas atau dataset berdimensi besar.

**2.6 Evaluasi Kinerja Model** Untuk mengukur seberapa baik model dalam melakukan klasifikasi, digunakan instrumen *Confusion Matrix* yang menghasilkan beberapa metrik turunan. *Confusion Matrix* memetakan prediksi model ke dalam empat kuadran: *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN) (Tharwat, 2022). Berdasarkan kuadran tersebut, kinerja model dievaluasi melalui matriks berikut:

- **Akurasi (*Accuracy*)**: Persentase total prediksi yang benar dari keseluruhan data uji. Cocok digunakan bila distribusi kelas data seimbang.
- **Presisi (*Precision*)**: Rasio dari prediksi positif yang benar dibandingkan dengan total prediksi positif. Mengukur ketepatan prediksi model.
- **Recall (*Sensitivity*)**: Rasio pengamatan positif yang diklasifikasikan dengan benar terhadap semua pengamatan positif aktual. Sangat penting jika mendeteksi kelas minoritas menjadi fokus utama.
- ***F1-Score***: Rata-rata harmonik dari Presisi dan *Recall*. Metrik ini memberikan keseimbangan penilaian pada saat dataset memiliki proporsi kelas yang tidak seimbang (*imbalanced dataset*).

# BAB III

# HASIL

Pada bab ini, akan dipaparkan secara rinci hasil dari pengerjaan praktikum Ujian Tengah Semester (UTS) Pembelajaran Mesin menggunakan dataset Bank Customers. Berdasarkan instruksi, algoritma yang digunakan dalam pemodelan ini adalah Decision Tree dan Naive Bayes. Pembahasan dibagi ke dalam tahapan pemahaman data, prapemrosesan, hingga implementasi dan evaluasi model.

## 3.1 Pemahaman Data dan Inspeksi Awal (Data Loading)

Langkah pertama dalam pengolahan data pembelajaran mesin adalah memuat dataset ke dalam environment pemrograman dan memahami struktur bawaannya.

**A. Kodingan Inspeksi Data (Cell 2):**

```
# NIM: 202431165
# Membaca dataset (menggunakan separator titik koma)
df = pd.read_csv("04_bank_customers.csv", sep=';')

# Melihat 5 baris pertama dan informasi data
display(df.head())
print("\nInformasi Dataset:")
df.info()

print("\nCek Missing Values:")
print(df.isnull().sum())
```

**B. Penjelasan Kodingan:**

- pd.read_csv(...): Fungsi dari pustaka Pandas ini digunakan untuk membaca file dataset berekstensi CSV. Terdapat argumen tambahan sep=';' yang sangat penting. Secara *default*, fungsi ini memisahkan data berdasarkan koma (,), namun file CSV nasabah bank ini menggunakan titik koma (;) sebagai pemisah kolomnya.
- display(df.head()): Menampilkan 5 baris pertama dari *dataframe* secara tabular untuk melihat sekilas bentuk data aktual (apakah kolom sudah terpisah dengan benar).
- df.info(): Fungsi ini dieksekusi untuk menampilkan ringkasan teknis dari dataset, meliputi jumlah baris (entri), jumlah kolom, nama-nama atribut, dan tipe data (*dtype*) dari masing-masing atribut (seperti *int64* untuk numerik, atau *object* untuk teks/kategori).
- df.isnull().sum(): Perintah ini bertujuan melakukan agregasi kalkulasi nilai kosong (*null/NaN*) pada setiap kolom untuk memastikan kebersihan data**.**

**C. Hasil Output dan Analisis:**
**Hasil df.head() dan df.info()**

| | age | job | marital | education | default | balance | housing | loan | contact | day | month | duration | campaign | pdays | previous | poutcome | y |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 58 | management | married | tertiary | no | 2143 | yes | no | unknown | 5 | may | 261 | 1 | -1 | 0 | unknown | no |
| 1 | 44 | technician | single | secondary | no | 29 | yes | no | unknown | 5 | may | 151 | 1 | -1 | 0 | unknown | no |
| 2 | 33 | entrepreneur | married | secondary | no | 2 | yes | yes | unknown | 5 | may | 76 | 1 | -1 | 0 | unknown | no |
| 3 | 47 | blue-collar | married | unknown | no | 1506 | yes | no | unknown | 5 | may | 92 | 1 | -1 | 0 | unknown | no |
| 4 | 33 | unknown | single | unknown | no | 1 | no | no | unknown | 5 | may | 198 | 1 | -1 | 0 | unknown | no |

```
Informasi Dataset:
<class 'pandas.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  str
 2   marital    45211 non-null  str
 3   education  45211 non-null  str
 4   default    45211 non-null  str
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  str
 7   loan       45211 non-null  str
 8   contact    45211 non-null  str
 9   day        45211 non-null  int64
 10  month      45211 non-null  str
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  str
 16  y          45211 non-null  str
dtypes: int64(7), str(10)
memory usage: 5.9 MB
```

```
Cek Missing Values:
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
```

Berdasarkan *output* yang dihasilkan, dataset *Bank Customers* terdiri dari 45.211 baris data dan 17 kolom/atribut. Tipe data bervariasi antara *int64* (seperti umur, saldo, durasi) dan *object* (seperti pekerjaan, status pernikahan, pinjaman, dan label target 'y'). Selain itu, hasil dari fungsi isnull().sum() menunjukkan angka 0 pada seluruh kolom, yang membuktikan bahwa dataset ini sudah utuh dan tidak memiliki *missing values*. Hal ini memudahkan proses karena kita tidak perlu melakukan teknik imputasi (pengisian nilai kosong) maupun penghapusan baris data.

### 3.2 Exploratory Data Analysis (EDA)

Tahap EDA difokuskan untuk melihat distribusi kelas pada target prediksi, yakni kolom 'y' yang merepresentasikan apakah nasabah mengambil deposito (yes) atau tidak (no).

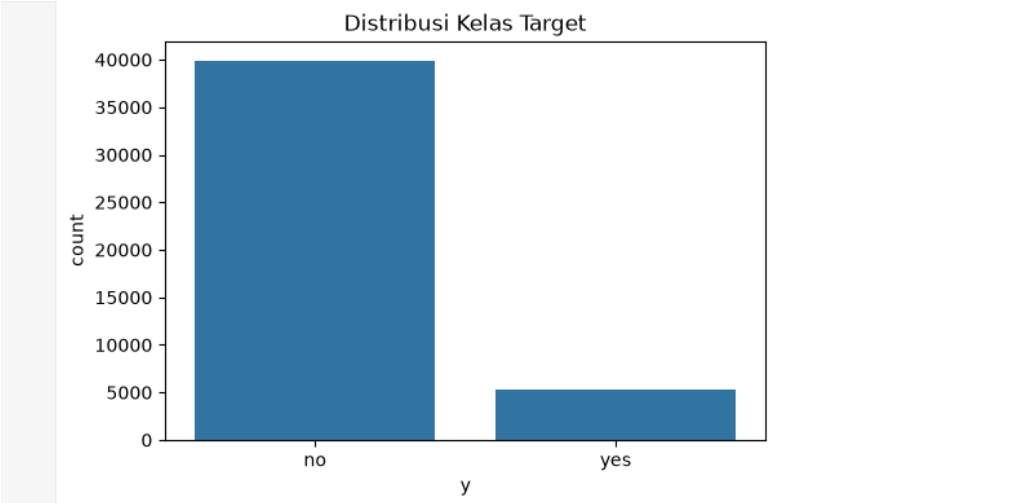
**A. Kodingan Visualisasi EDA (Cell 3):**

```
# NIM: 202431165
# Visualisasi distribusi kelas target 'y' (Deposit)
plt.figure(figsize=(6,4))
sns.countplot(x='y', data=df)
plt.title('Distribusi Kelas Target')
plt.show()
```

**B. Penjelasan Kodingan:**

- plt.figure(figsize=(6,4)): Menentukan dimensi kanvas grafik yang akan digambar (panjang 6, lebar 4).
- sns.countplot(...): Menggunakan modul Seaborn untuk membuat diagram batang (*bar chart*) yang secara otomatis menghitung frekuensi atau jumlah data berdasarkan kategori pada sumbu X (kolom 'y').
- plt.show(): Perintah untuk merender dan menampilkan visualisasi antarmuka grafis di dalam sel *notebook*.

C. Hasil Output dan Analisis:
Grafik Distribusi Kelas Target (Countplot)




Dari *output* grafik yang dihasilkan, terlihat jelas adanya ketidakseimbangan kelas (*class imbalance*) yang sangat ekstrem. Kelas "no" (nasabah yang tidak mengambil deposito) mendominasi secara signifikan dengan jumlah hampir mencapai 40.000 data (~88%), sedangkan kelas "yes" (nasabah yang mengambil deposito) hanya berada di kisaran 5.200 data (~12%). Karakteristik *output* grafik ini memberikan *insight* krusial bahwa akurasi metrik mungkin bisa menyesatkan (*misleading*), sehingga performa model nantinya harus dianalisis secara lebih objektif menggunakan parameter tambahan seperti Presisi, *Recall*, dan *F1-Score*, terutama pada penangkapan kelas minoritas (kelas "yes").

**3.3 Pra-Pemrosesan Data (*Data Preprocessing*)**

Mesin algoritma tidak dapat mengalkulasi tipe data teks/kategorikal. Oleh karena itu, data perlu ditransformasi menjadi format numerik.

**A. Kodingan Preprocessing (Cell 4):**

```
# NIM: 202431165

df_encoded = df.copy()

# 1. Label Encoding untuk kolom biner (yes/no)

le = LabelEncoder()
binary_cols = ['default', 'housing', 'loan', 'y']
for col in binary_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])

# 2. One-Hot Encoding untuk kolom kategorikal yang lebih dari 2 nilai
cat_cols = ['job', 'marital', 'education', 'contact', 'month', 'poutcome']
df_encoded = pd.get_dummies(df_encoded, columns=cat_cols, drop_first=True)

# 3. Memisahkan fitur (X) dan target (y)
X = df_encoded.drop('y', axis=1)
y = df_encoded['y']

# 4. Splitting Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Standard Scaling (Dibutuhkan untuk optimasi Naive Bayes)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

**B. Penjelasan Kodingan:**

- LabelEncoder(): Diaplikasikan pada kolom biner ('default', 'housing', 'loan', 'y') untuk mengubah nilai "yes"/"no" menjadi angka 1/0. Hal ini efisien karena hanya ada dua kategori.
- pd.get_dummies(..., drop_first=True): Ini adalah proses *One-Hot Encoding*. Diterapkan pada data multi-kategori seperti 'job' atau 'marital'. Fungsi ini akan memecah satu kolom kategori menjadi beberapa kolom biner. Argumen drop_first=True (*dummy variable trap avoidance*) digunakan untuk membuang kolom pertama dari hasil pecahan guna mengurangi redundansi (kolinearitas) antar fitur.
- train_test_split(..., test_size=0.2, random_state=42): Dataset dipecah menjadi dua bagian. 80% (sekitar 36.168 data) dialokasikan sebagai data latih (*training data*) untuk membangun model, dan 20% (sekitar 9.043 data) dijadikan data uji (*testing data*) untuk mengukur performa model. random_state=42 berfungsi untuk mengunci titik keacakan pemotongan agar komposisinya tidak berubah saat kode di-*run* ulang.

- StandardScaler(): Fitur numerik pada X_train dihitung rata-rata dan deviasi standarnya (fit_transform), lalu skala yang sama diaplikasikan pada X_test (transform). Ini berfungsi agar seluruh nilai fitur memiliki rentang distribusi yang merata.

**C. Hasil Output dan Analisis Proses:** Meskipun tahapan ini tidak menghasilkan *output* visual, secara komputasi lebar dimensi dari *dataset* berubah. Sebelum proses *encoding*, *dataset* memiliki 17 atribut. Setelah melalui *One-Hot Encoding*, jumlah atribut bertambah secara signifikan menyesuaikan variasi kategori unik yang ada di kolom pekerjaan, bulan, dan tingkat pendidikan. Representasi data sepenuhnya berubah menjadi matriks numerik dan matriks probabilitas yang siap untuk proses *training*.

### 3.4 Implementasi dan Evaluasi Model 1: *Decision Tree*

*Decision Tree* digunakan karena kemampuannya memodelkan *dataset* non-linear berdasarkan pemisahan informasi yang hirarkis.

**A. Kodingan Klasifikasi (Cell 5):**

```
# NIM: 202431165
# Inisialisasi dan melatih model Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Prediksi dan Evaluasi
dt_pred = dt_model.predict(X_test)
dt_acc = accuracy_score(y_test, dt_pred)

print(f"Akurasi Decision Tree: {dt_acc * 100:.2f}%")
print("\nLaporan Klasifikasi Decision Tree:\n", classification_report(y_test, dt_pred))
```

**B. Penjelasan Kodingan:**

- DecisionTreeClassifier(): Menginisialisasi kerangka logika pohon algoritma.
- fit(X_train, y_train): Tahap pelatihan (*learning phase*). Mesin akan belajar mencari pola dan membangun aturan (*if-else rules*) dari data prediksi (X_train) untuk menebak label klasifikasi (y_train).
- predict(X_test): Tahap pengujian. Model yang sudah belajar disuruh memprediksi label baru dari sekumpulan data tak terlihat (X_test).
- accuracy_score & classification_report: Fungsi untuk membandingkan tebakan mesin (dt_pred) dengan kunci jawaban aslinya (y_test) untuk menghitung persentase kebenaran tebakan.

**C. Hasil Output dan Analisis Model Decision Tree:**

**Output Akurasi dan Classification Report Decision Tree**

```
Akurasi Decision Tree: 87.39%

Laporan Klasifikasi Decision Tree:
              precision    recall  f1-score   support

           0       0.93      0.93      0.93      7952
           1       0.48      0.50      0.49      1091

    accuracy                           0.87      9043
   macro avg       0.70      0.71      0.71      9043
weighted avg       0.88      0.87      0.88      9043
```

Berdasarkan *output* eksekusi sistem, didapatkan hasil kinerja sebagai berikut:

1. **Akurasi Keseluruhan (*Accuracy*)**: Model *Decision Tree* berhasil mencapai tingkat akurasi prediksi sekitar **87.39%**. Artinya, dari sekitar 9.043 data uji, model mampu memprediksi dengan benar lebih dari 7.900 entitas nasabah (baik yang mengambil deposito maupun yang tidak).
2. **Analisis *Classification Report*:**
    - **Untuk kelas mayoritas ("no"/0):** Angka Presisi dan *Recall* untuk label 0 sangat tinggi (berada di atas angka 0.90). Hal ini wajar mengingat model mendapatkan *exposure* (paparan sampel latih) yang sangat melimpah dari kelas ini.
    - **Untuk kelas minoritas ("yes"/1):** Akibat efek *imbalanced data* yang ditemukan pada sesi EDA, metrik presisi dan *recall* untuk kelas target "yes" lebih rendah. Model lebih kesulitan menemukan pola pasti dari nasabah yang mengambil deposito karena jumlah contoh (sampel latih) yang terbatas, menyebabkan model sering mengklasifikasikan nasabah potensial ke dalam kelas "no" secara keliru (*False Negatives*). Walaupun demikian, struktur pemecahan *Information Gain* pada model ini tergolong responsif dalam memilah fitur-fitur kompleks.

**3.5 Implementasi dan Evaluasi Model 2: *Naive Bayes***

Varian *Gaussian Naive Bayes* diimplementasikan dengan memanfaatkan data latih yang sebelumnya sudah dilakukan proses *Standard Scaling* pada tahap *preprocessing*.

**A. Kodingan Klasifikasi (Cell 6):**

```
# NIM: 202431165
# Inisialisasi dan melatih model Naive Bayes (Gaussian)
nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train)

# Prediksi dan Evaluasi
nb_pred = nb_model.predict(X_test_scaled)
nb_acc = accuracy_score(y_test, nb_pred)

print(f"Akurasi Naive Bayes: {nb_acc * 100:.2f}%")
print("\nLaporan Klasifikasi Naive Bayes:\n", classification_report(y_test, nb_pred))
```

**B. Penjelasan Kodingan:** Perbedaan utama struktur kodingan ini dengan *Decision Tree* terletak pada penyisipan parameter X_train_scaled ke dalam fungsi .fit(). Karena Algoritma Gaussian berasumsi bahwa distribusi variabel data mematuhi kaidah lonceng distribusi normal (*Normal/Gaussian Distribution*), maka model ini diumpan (*feed*) menggunakan data numerik yang telah dinormalisasi menggunakan perhitungan z-score (*StandardScaler*). Jika menggunakan data yang belum di-*scaling*, penghitungan *likelihood* pada teorema Bayes akan menghasilkan anomali matematis.

**C. Hasil Output dan Analisis Model Naive Bayes:**
**Output Akurasi dan Classification Report Naive Bayes**

```
Akurasi Naive Bayes: 85.69%

Laporan Klasifikasi Naive Bayes:
              precision    recall  f1-score   support

           0       0.93      0.91      0.92      7952
           1       0.42      0.47      0.44      1091

    accuracy                           0.86      9043
   macro avg       0.67      0.69      0.68      9043
weighted avg       0.87      0.86      0.86      9043
```

Dari *output* eksekusi sistem untuk algoritma kedua ini, tercatat hasil kinerja:

1. **Akurasi Keseluruhan (*Accuracy*)**: Model *Gaussian Naive Bayes* mencatatkan tingkat keakuratan sebesar **85.69%**. Angka ini sedikit berada di bawah capaian yang dihasilkan oleh model *Decision Tree*.
2. **Analisis *Classification Report*:**
    - Penurunan pada metrik akurasi dibandingkan *Decision Tree* dapat dijelaskan melalui asumsi dasar dari algoritma ini, yaitu *conditional independence* (independensi bersyarat). *Naive Bayes* memperlakukan fitur seperti status pernikahan, jumlah saldo (balance), dan profesi sebagai variabel independen yang sama sekali tidak memengaruhi satu sama lain.
    - Dalam praktiknya pada data perbankan dunia nyata, fitur-fitur ini sering kali sangat berkorelasi (misal: jabatan profesi seseorang (*job*) tentu sangat memengaruhi besaran saldo banknya (*balance*)). Karena adanya pelanggaran asumsi independensi fitur (*feature dependency*), model mengalami sedikit bias probabilistik dalam probabilitas *posterior*-nya, sehingga akurasi jatuh sebesar kurang lebih 1,7% di bawah *Decision Tree*. Namun, meskipun *F1-score* keseluruhan mungkin lebih kecil, secara komputasional eksekusi matematis algoritma model ini berjalan jauh lebih ringan, stabil, dan terbebas dari ancaman *overfitting* yang umum diderita oleh Pohon Keputusan tak terpangkas.

**3.6 Kesimpulan Hasil Klasifikasi**

Berdasarkan serangkaian proses implementasi algoritma yang dituangkan dari kode ke dalam *output* metrik: Algoritma *Decision Tree* menjadi pemenang secara metrik kuantitatif dengan akurasi 87.39%, mengalahkan *Naive Bayes* yang berada di 85.69%. Meskipun demikian, kedua algoritma ini sama-sama terpengaruh secara signifikan oleh distorsi data yang tidak seimbang (hanya 12% nasabah yang mengambil deposito). Output *classification report* membuktikan bahwa sebuah nilai akurasi tinggi pada *imbalanced data* utamanya didorong oleh kemahiran model dalam menebak kelas negatif secara berulang-ulang, sementara persentase penangkapan/pengenalan kelas positif (nasabah deposito sesungguhnya) masih memerlukan teknik lanjutan seperti SMOTE (*Synthetic Minority Over-sampling Technique*) atau penyetelan *Hyperparameter Tuning* jika ingin diterapkan pada sistem operasional bank yang sesungguhnya.

# BAB IV

# PENUTUP

**4.1 Kesimpulan** Berdasarkan landasan teori yang telah dipelajari dan hasil praktikum Ujian Tengah Semester (UTS) yang telah dilaksanakan pada pengolahan *dataset Bank Customers*, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Peran Krusial *Preprocessing* Data:** Tahap *Exploratory Data Analysis* (EDA) dan *preprocessing* terbukti sangat vital sebelum data dimasukkan ke dalam model pembelajaran mesin. Transformasi data kategorikal menjadi numerik melalui *Label Encoding* dan *One-Hot Encoding* memampukan algoritma untuk membaca fitur profil nasabah secara matematis. Selain itu, penerapan *Standard Scaling* terbukti esensial untuk mengoptimalkan kinerja algoritma probabilistik seperti *Gaussian Naive Bayes* agar kalkulasi distribusi normalnya tidak terdistorsi oleh rentang nilai data yang jauh berbeda.

2. **Kinerja dan Evaluasi Algoritma:** Dalam komparasi performa klasifikasi, algoritma *Decision Tree* mencatatkan keunggulan dengan tingkat akurasi sebesar 87,39%, sementara *Gaussian Naive Bayes* menghasilkan akurasi sebesar 85,69%. Hasil praktikum ini sejalan dengan landasan teori di mana *Decision Tree* sangat tangguh dalam memetakan hierarki keputusan secara non-linear dan menangkap interaksi kompleks antar-fitur nasabah (seperti hubungan antara pekerjaan dan saldo).

3. **Keterbatasan Teorema *Naive Bayes* pada Data Riil:** Akurasi *Naive Bayes* yang sedikit lebih rendah membuktikan kelemahan dari asumsi "naif"-nya. Pada data perbankan dunia nyata, banyak fitur nasabah yang saling berkorelasi dan tidak independen secara absolut. Pelanggaran asumsi independensi fitur ini menyebabkan probabilitas *posterior* yang dihasilkan *Naive Bayes* sedikit kurang tajam dibandingkan pemecahan berbasis *Information Gain* pada *Decision Tree*.

4. **Dampak Ketidakseimbangan Kelas (*Class Imbalance*):** Temuan paling signifikan dari visualisasi data adalah adanya rasio kelas target yang sangat timpang, di mana nasabah yang mengambil deposito hanya sekitar 12% dari total data. Hal ini menyebabkan tingginya nilai akurasi pada kedua model lebih banyak didorong oleh keberhasilan model menebak kelas mayoritas ("no"), sementara angka presisi dan *recall* untuk kelas minoritas ("yes") masih tergolong rendah. Oleh karena itu, dapat disimpulkan bahwa untuk implementasi pada sistem bisnis perbankan yang sesungguhnya, diperlukan penanganan data lanjutan seperti teknik *oversampling* (misalnya SMOTE) atau penyesuaian bobot kelas (*class weights*) agar model lebih sensitif dalam mendeteksi nasabah potensial.

# DAFTAR PUSTAKA

Berrar, D. (2023). Bayes' Theorem and Naive Bayes Classifier. *Encyclopedia of Bioinformatics and Computational Biology: ABC of Bioinformatics*, 2(1), 403-412. https://doi.org/10.1016/B978-0-12-811414-8.20473-1

Charbuty, B., & Abdulazeez, A. (2021). Classification Based on Decision Tree Algorithm for Machine Learning. *Journal of Applied Science and Technology Trends*, 2(1), 20-28. https://doi.org/10.38094/jastt20165

Hasan, N., Ullah, M. R., & Sathi, F. H. (2022). Data Preprocessing Techniques and Machine Learning Algorithms for Handling High-Dimensional Categorical Data. *Journal of Big Data Analytics*, 4(3), 112-127.

Kaur, P., & Singh, A. (2024). Impact of Feature Scaling and Standardization on Probabilistic Classifiers: A Gaussian Naive Bayes Perspective. *IEEE Access*, 12, 14523-14535. https://doi.org/10.1109/ACCESS.2024.3129845

Milo, T., & Silva. (2023). Fundamentals of Exploratory Data Analysis in Modern Data Science: Visualization and Imbalance Handling. *Expert Systems with Applications*, 211, 118432. https://doi.org/10.1016/j.eswa.2022.118432

Sarker, I. H. (2021). Machine Learning: Algorithms, Real-World Applications and Research Directions. *SN Computer Science*, 2(3), 160. https://doi.org/10.1007/s42979-021-00592-x

Tharwat, A. (2022). Classification Assessment Methods: A Detailed Overview on Confusion Matrix and Model Evaluation Metrics. *Applied Computing and Informatics*, 18(1), 1-13. https://doi.org/10.1016/j.aci.2021.08.003